ASSIGNMENT 02

NAME: DIYA KUMAL

ROLL NO: 2501720001

COURSE: BSC COMPUTER SCIENCE

1. Dynamic Array – Amortized Analysis

A dynamic array increases its size when it becomes full. Initially, it has a fixed capacity. When the array is full and a new element is added, a new array of double capacity is created and all elements are copied.

- Most append() operations take **O(1)** time because they simply add an element.
- When resizing happens, it takes **O(n)** time because all elements are copied.
- However, resizing does not happen every time.

Therefore, the **amortized time complexity of append() is O(1)**.

2. Time Complexity of Operations

Stack (Using Linked List)

- push() → O(1)
- pop() → O(1)
- peek() → O(1)

Reason: Operations happen at the head.

Queue (Using Linked List)

- enqueue() → O(1)
- dequeue() → O(1)
- front() → O(1)

Reason: Uses both head and tail pointers

3. Explanation of Parentheses Checker

The stack is used to check whether parentheses are balanced.

- Push opening brackets (, {, [
- When a closing bracket comes:
  - Pop from stack
  - Check if it matches

If stack is empty at the end → Balanced
Otherwise → Not Balanced

4. Conclusion

In this assignment, we implemented linear data structures like arrays, linked lists, stacks, and queues. We also applied the stack to solve a real-life problem (parentheses checking). This helped in understanding both implementation and practical usage.